

Introduction

Welcome! In this assignment, you will build a Convolutional Neural Network (CNN) to classify handwritten digits from the famous MNIST dataset. This dataset is a classic in the field of computer vision and provides a great starting point for understanding image classification with deep learning.

This notebook is structured to guide you step-by-step through the process. You will load the data, preprocess it, define a CNN model, train it, and evaluate its performance. Throughout the assignment, you will have opportunities to experiment and deepen your understanding of the concepts.

Remember to:

- **Read all instructions carefully.**
- **Execute the code cells in order.**
- **Fill in the missing code sections marked as "Students: Fill in the blanks".**
- **Answer the reflection questions in the designated Markdown cell.**
- **Experiment and explore!** Change parameters, layers, and observe the effects.

Let's get started and build our MNIST digit classifier!

Section 1: Setting Up - Imports

Before we dive into building our CNN, we need to import the necessary libraries. These libraries provide pre-built tools and functions that will make our work much easier.

Instructions:

1. **Carefully review the code cell below.** It imports libraries from TensorFlow and Keras, which are powerful frameworks for building and training neural networks.
2. **Execute the code cell by selecting it and pressing [Shift + Enter] (or the "Run" button).**
3. **Ensure there are no error messages after running the cell.** If you encounter errors, double-check that you have TensorFlow and Keras installed in your environment.

```
# Cell 1: Imports

import tensorflow as tf
```

```
from tensorflow.keras.datasets import mnist

Role: Provides convenient access to handwritten-digit images
used to train and evaluate an image-classification model.
Status: READY – digit archive connection established

"""

print("""

Label Translator

from tensorflow.keras.utils import to_categorical

Role: Converts a label such as 3 into a one-hot vector:

3 -> [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]

Status: READY – labels can be prepared for classification

""")

print("=" * 74)
print("IMPORT MISSION COMPLETE")
print("The research lab is equipped to load digits, build a CNN.")
print("train it, and translate predictions into digit classes.")
print("=" * 74)
```

```
Role: The computational engine that performs tensor math,
manages hardware acceleration, and powers deep-learning tasks.
Status: ONLINE – TensorFlow 2.20.0

Keras Model Architect

from tensorflow import keras

Role: TensorFlow's high-level interface for designing, compiling,
training, and evaluating neural-network models.
Status: READY – model construction tools unlocked
```

```
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# Visual import explanation

print("=" * 74)
print("NEURAL NETWORK RESEARCH LAB")
print("Import Crew: Online")
print("=" * 74)

print(f"""

TensorFlow Engine

import tensorflow as tf

Role: The computational engine that performs tensor math,
manages hardware acceleration, and powers deep-learning tasks.
Status: ONLINE – TensorFlow {tf.__version__}

""")

print("""

Keras Model Architect

from tensorflow import keras

Role: TensorFlow's high-level interface for designing, compiling,
training, and evaluating neural-network models.
Status: READY – model construction tools unlocked

""")

print("""

Layers Construction Team

from tensorflow.keras import layers

Role: Supplies neural-network building blocks such as Dense,
Conv2D, MaxPooling2D, Flatten, and Dropout layers.
Status: READY – architecture components available

""")

print("""

MNIST Digital Archive
```

```
Conv2D, MaxPooling2D, Flatten, and Dropout layers.
Status: READY – architecture components available

MNIST Digital Archive

from tensorflow.keras.datasets import mnist

Role: Provides convenient access to handwritten-digit images
used to train and evaluate an image-classification model.
Status: READY – digit archive connection established

Label Translator

from tensorflow.keras.utils import to_categorical

Role: Converts a label such as 3 into a one-hot vector:

3 -> [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]

Status: READY – labels can be prepared for classification
```

```
IMPORT MISSION COMPLETE
The research lab is equipped to load digits, build a CNN,
train it, and translate predictions into digit classes.
```

```
# Imports and visual status displays

import os
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

print("TensorFlow version:", tf.__version__)
print("Checking for the MNIST dataset...")

# Default Keras cache location
cache_dir = os.path.join(os.path.expanduser("~"), ".keras", "datasets")
os.makedirs(cache_dir, exist_ok=True)

mnist_path = os.path.join(cache_dir, "mnist.npz")

if os.path.exists(mnist_path):
    size_mb = os.path.getsize(mnist_path) / (1024 * 1024)
```

```
print(f" MNIST dataset already downloaded.")
print(f" File: {mnist_path}")
print(f" Size: {size_mb:.2f} MB")
else:
    print("⚠ Downloading MNIST dataset...")
    print(" A progress bar should appear below.\n")

    # keras.utils.get_file displays a download progress bar
    mnist_path = keras.utils.get_file(
        fname="mnist.npz",
        origin="https://storage.googleapis.com/tensorflow/tf-keras-datas
        cache_dir=os.path.expanduser("~"),
        cache_subdir=".keras/datasets"
    )

    size_mb = os.path.getsize(mnist_path) / (1024 * 1024)
    print(f"\n Download complete.")
    print(f" Saved to: {mnist_path}")
    print(f" Size: {size_mb:.2f} MB")

print("\nLoading MNIST into memory...")

# Load the locally downloaded file
(x_train, y_train), (x_test, y_test) = mnist.load_data()

print(" MNIST loaded successfully!")
print(f"Training images: {x_train.shape}")
print(f"Training labels: {y_train.shape}")
print(f"Test images: {x_test.shape}")
print(f"Test labels: {y_test.shape}")

import os
import time
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.datasets import mnist

def pause(seconds=0.7):
    time.sleep(seconds)

def divider():
    print("=" * 68)

def status(icon, title, message=""):
    print(f"\n{icon} {title}")
    if message:
        print(f" {message}")

# -----
# 1. Opening scene: TensorFlow starts
# -----
```

```
# -----
# 3. Load dataset
# -----

status("📁", "Opening the MNIST archive...")
pause()

(x_train, y_train), (x_test, y_test) = mnist.load_data()

print(r"""


| HANDWRITING RESEARCH LAB |   |   |                |   |   |   |   |  |
|--------------------------|---|---|----------------|---|---|---|---|--|
| 0                        | 1 | 2 | 3              | 4 | 5 | 6 | 7 |  |
| 8                        | 9 | → | Neural Network |   |   |   |   |  |


""")

pause()

# -----
# 4. Dataset inventory
# -----

divider()
print(" DATASET INVENTORY")
divider()

print(f"""


| Collection        | Image Count             | Image Format        |
|-------------------|-------------------------|---------------------|
| Training workshop | {x_train.shape[0]:>10,} | {x_train.shape[1]}  |
| Final examination | {x_test.shape[0]:>10,}  | {x_test.shape[1]} × |


""")

print(" Label classes: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9")
print(f" Training labels: {y_train.shape[0]:,}")
print(f" Test labels: {y_test.shape[0]:,}")

divider()
print(" MISSION COMPLETE: 70,000 handwritten digits are ready for trai
print(" Next stage: normalize pixels and build the neural network.")
divider()
```

```
divider()
print(" MNIST DIGIT ARCHIVE")
print(" Neural Network Loading Bay")
divider()

status("⚙️", "TensorFlow engine online", f"Version {tf.__version__}")
pause()

# -----
# 2. Inspect Keras cache / vault
# -----

cache_dir = os.path.join(os.path.expanduser("~"), ".keras", "datasets")
mnist_path = os.path.join(cache_dir, "mnist.npz")

status("🔍", "Searching the Keras data vault...")
pause()

if os.path.exists(mnist_path):
    size_mb = os.path.getsize(mnist_path) / (1024 * 1024)

    print(r"""


```

graph TD
 A[DATA VAULT FOUND] --> B[MNIST ARCHIVE]

```


""")

    status("✅", "Archive recovered from local storage")
    print(f" Location: {mnist_path}")
    print(f" Weight: {size_mb:.2f} MB")

else:
    status("❌", "MNIST archive is missing – requesting delivery...")
    print("\n Satellite link: [REDACTED] 0%")
    pause()

    keras.utils.get_file(
        fname="mnist.npz",
        origin="https://storage.googleapis.com/tensorflow/tf-keras-datas
        cache_dir=os.path.expanduser("~"),
        cache_subdir=".keras/datasets"
    )

    print("\n Satellite link: [REDACTED] 100%")
    status("Archive delivered and secured in the data vault")
```

```
Test images: (10000, 28, 28)
Test labels: (10000,)

# -----
# MNIST DIGIT ARCHIVE
# Neural Network Loading Bay
# -----

⚙️ TensorFlow engine online
Version 2.20.0

🔍 Searching the Keras data vault...



```

graph TD
 A[DATA VAULT FOUND] --> B[MNIST ARCHIVE]

```



✅ Archive recovered from local storage
Location: /root/.keras/datasets/mnist.npz
Weight: 10.96 MB

📁 Opening the MNIST archive...



| HANDWRITING RESEARCH LAB |   |   |                |   |   |   |   |  |
|--------------------------|---|---|----------------|---|---|---|---|--|
| 0                        | 1 | 2 | 3              | 4 | 5 | 6 | 7 |  |
| 8                        | 9 | → | Neural Network |   |   |   |   |  |



# -----
# DATASET INVENTORY
# -----



| Collection        | Image Count | Image Format             |
|-------------------|-------------|--------------------------|
| Training workshop | 60,000      | 28 × 28 grayscale pixels |
| Final examination | 10,000      | 28 × 28 grayscale pixels |



Label classes: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Training labels: 60,000
Test labels: 10,000

MISSION COMPLETE: 70,000 handwritten digits are ready for training.
Next stage: normalize pixels and build the neural network.
```

Explanation of Imports:

- `(tensorflow as tf and keras)`: TensorFlow is the main deep learning framework, and Keras is its high-level API that simplifies building and training models. We import TensorFlow as `(tf)` and Keras directly for easy access to their functionalities.
- `(from tensorflow.keras import layers)`: This imports the `(layers)` module from Keras, which provides various layers for building neural networks (like convolutional layers, dense layers, etc.).
- `(from tensorflow.keras.datasets import mnist)`: This imports the MNIST dataset directly from Keras datasets. This is very convenient for loading and using the MNIST data.
- `(from tensorflow.keras.utils import to_categorical)`: This imports the `(to_categorical)` function, which we will use to perform one-hot encoding of our labels.

Section 2: Data Loading and Preprocessing

In this section, we will load the MNIST dataset and prepare it for training our CNN model. Preprocessing steps are crucial to ensure our data is in the right format for the model to learn effectively.

Instructions:

1. **Read through the code in the cell below.** Understand how it loads the MNIST dataset and what preprocessing steps are applied.
2. **Execute the code cell.**
3. **Examine the comments in the code** to understand each preprocessing step in detail.

```
# Cell 2: Data Loading and Preprocessing
# Neural Network Data Preparation Pipeline
```

```
import matplotlib.pyplot as plt
import numpy as np
```

```
def divider():
    print("=" * 72)
```

```
def stage(title, message):
    print(f"""
```

```
| {title:<66} |
|-----|
| {message:<66} |
```

```
# -----
# Stage 2: Normalization
# -----
```

```
stage(
    "STAGE 02 – PIXEL NORMALIZATION",
    "Converting human image values into neural network values"
)
```

```
print("""
Before:
```

```
Pixel Range:

0 ----- 255
```

```
Transformation:
```

```
pixel / 255
```

```
After:
```

```
0.0 ----- 1.0
```

```
Purpose:
```

```
The CNN learns more efficiently when inputs
are placed into a consistent numerical range.
""")
```

```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

```
print(f"""
Normalized Training Range:
```

```
Minimum Pixel Value: {x_train.min()}
Maximum Pixel Value: {x_train.max()}
""")
```

```
plt.figure(figsize=(3,3))
plt.imshow(x_train[sample], cmap="gray")
plt.title("NORMALIZED IMAGE\nCNN Ready Pixel Values")
```

```
""")
```

```
divider()
print("""
                CNN DATA PREPARATION CENTER
                Image Conversion & Feature Preparation
""")
divider()
```

```
# -----
# Stage 1: Raw Dataset
# -----
```

```
stage(
    "STAGE 01 – RAW IMAGE COLLECTION",
    "Loading handwritten images before neural processing"
)
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
print(f"""
Incoming Training Data:
```

```
Images : {x_train.shape}
Labels : {y_train.shape}
```

```
Image Type:
Height : 28 pixels
Width : 28 pixels
Color : Grayscale (1 channel)
```

```
Pixel Information:
```

```
Dark Pixel = 0
Bright Pixel = 255
```

```
""")
```

```
# Display original image
```

```
sample = 0
```

```
plt.figure(figsize=(3,3))
plt.imshow(x_train[sample], cmap="gray")
plt.title(f"RAW INPUT IMAGE\nDigit Label: {y_train[sample]}")
plt.axis("off")
plt.show()
```

```
plt.axis("off")
plt.show()
```

```
# -----
# Stage 3: CNN Shape Conversion
# -----
```

```
stage(
    "STAGE 03 – IMAGE DIMENSION CONVERSION",
    "Adding channel information required by Conv2D layers"
)
```

```
print("""
Before CNN formatting:
```

```
(Images, Height, Width)

(60000, 28, 28)
```

```
After CNN formatting:
```

```
(Images, Height, Width, Channels)

(60000, 28, 28, 1)
```

```
Channel Explanation:
```

```
1 = grayscale image channel
""")
```

```
x_train = x_train.reshape(-1,28,28,1)
x_test = x_test.reshape(-1,28,28,1)
```

```
print("New Training Shape:")
print(x_train.shape)
```

```
# -----
# Stage 4: Label Encoding
# -----
```

```
stage(
    "STAGE 04 – LABEL TRANSLATION",
```

```

    "Converting digit answers into neural network format"
)

print("""
Original Human Label:

    7

One-Hot Neural Representation:

    [0,0,0,0,0,0,0,1,0,0]

Why?

The CNN compares probability across
all possible digit classes.
""")

num_classes = 10

y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)

print("Example Encoded Label:")
print(y_train[sample])

# -----
# Completion
# -----

divider()

print("""
        DATA PREPARATION COMPLETE

        IMAGE → NORMALIZED PIXELS → CNN TENSOR
                |
                ↓
        ONE-HOT CLASS LABEL

The dataset is now prepared for CNN training.
""")

divider()

```

CNN DATA PREPARATION CENTER

Image Conversion & Feature Preparation

STAGE 01 – RAW IMAGE COLLECTION

Loading handwritten images before neural processing

Incoming Training Data:

Images : (60000, 28, 28)
Labels : (60000,)

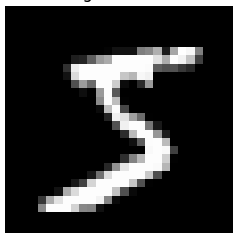
Image Type:

Height : 28 pixels
Width : 28 pixels
Color : Grayscale (1 channel)

Pixel Information:

Dark Pixel = 0
Bright Pixel = 255

RAW INPUT IMAGE
Digit Label: 5



STAGE 02 – PIXEL NORMALIZATION

Converting human image values into neural network values

Before:

Pixel Range:

0 ————— 255

Transformation:

pixel / 255

After:

0.0 ————— 1.0

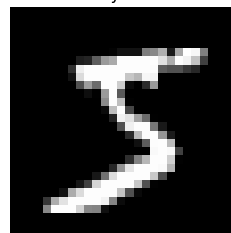
Purpose:

The CNN learns more efficiently when inputs are placed into a consistent numerical range.

Normalized Training Range:

Minimum Pixel Value: 0.0
Maximum Pixel Value: 1.0

NORMALIZED IMAGE
CNN Ready Pixel Values



STAGE 03 – IMAGE DIMENSION CONVERSION

Adding channel information required by Conv2D layers

Before CNN formatting:

(Images, Height, Width)

(60000, 28, 28)

After CNN formatting:

(Images, Height, Width, Channels)

(60000, 28, 28, 1)

Channel Explanation:

1 = grayscale image channel

New Training Shape:

(60000, 28, 28, 1)

STAGE 04 – LABEL TRANSLATION

Converting digit answers into neural network format

Original Human Label:

7

One-Hot Neural Representation:

[0,0,0,0,0,0,0,1,0,0]

Why?

The CNN compares probability across all possible digit classes.

Example Encoded Label:

[0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]

DATA PREPARATION COMPLETE

IMAGE → NORMALIZED PIXELS → CNN TENSOR

↓

ONE-HOT CLASS LABEL

The dataset is now prepared for CNN training.

3. **Experiment!** You are encouraged to try different configurations for the layers, such as changing the number of filters in the convolutional layers, or adding more layers.

Cell 3: CNN Model Definition
Neural Network Architecture Construction Lab

```
from tensorflow import keras
from tensorflow.keras import layers
```

```
def divider():
    print("=" * 76)
```

```
def module(title, description):
    print(f"""
```

```
| {title:<74} |
```

```
| {description:<74} |
```

```
""")
```

```
divider()
```

```
print("""
```

```
    CNN ARCHITECTURE CONSTRUCTION LAB
```

```
    Building the feature detection system
    that will learn handwritten digits
```

```
""")
```

```
divider()
```

```
# -----
# Input Layer
# -----
```

```
module(
    "MODULE 01 – IMAGE INPUT SYSTEM",
    "Receiving prepared MNIST image tensors"
)
```

```
print("""
Incoming Data:
```

```
    Image Shape:
```

```
        Height      Width      Channel
```

Explanation of Data Preprocessing:

- **Loading the MNIST dataset:** `mnist.load_data()` loads the MNIST dataset, which is already split into training and testing sets (`(x_train, y_train)`, `(x_test, y_test)`). `x_train` and `x_test` contain the images (pixel data), and `y_train` and `y_test` contain the corresponding labels (digits 0-9).
- **Normalization:** `x_train = x_train.astype("float32") / 255.0` and `x_test = x_test.astype("float32") / 255.0` normalize the pixel values. Pixel values in images are typically in the range 0-255. Dividing by 255 scales them to the range 0-1. This normalization helps the neural network train faster and more effectively.
- **Adding Channel Dimension:** `x_train = x_train.reshape(-1, 28, 28, 1)` and `x_test = x_test.reshape(-1, 28, 28, 1)` reshape the data to add a channel dimension. Even though MNIST images are grayscale (single channel), CNNs in Keras expect input data to have a channel dimension. We reshape from `(number_of_images, 28, 28)` to `(number_of_images, 28, 28, 1)`. The `-1` in `reshape` means "infer the dimension based on the size of the array."
- **One-Hot Encoding:** `y_train = to_categorical(y_train, num_classes)` and `y_test = to_categorical(y_test, num_classes)` perform one-hot encoding on the labels. Instead of representing the digit '3' as a single number, one-hot encoding converts it into a vector `[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]`, where the 4th position (index 3) is 'hot' (value 1), and all other positions are 'cold' (value 0). This is a standard way to represent categorical labels for neural networks in multi-class classification problems. `num_classes = 10` specifies that we have 10 classes (digits 0-9).

Section 3: Model Definition - Building the CNN

Now we will define the architecture of our Convolutional Neural Network (CNN). You will be building a sequential model using Keras layers.

Instructions:

1. **Carefully examine the code in the cell below.** Notice the structure of the `keras.Sequential` model.
2. **Fill in the missing parts** marked with `# Students: Fill in the blanks` to complete the model definition.

28 x 28 x 1

The CNN receives grayscale images prepared in the previous stage.

```
""")
```

```
# -----
# Build CNN Architecture
# -----
```

```
module(
    "MODULE 02 – FEATURE EXTRACTION ENGINE",
    "Creating convolution and pooling layers"
)
```

```
model = keras.Sequential(
[
    # Input
    keras.Input(shape=(28,28,1)),
```

```
    # First convolution
    layers.Conv2D(
        32,
        kernel_size=(3,3),
        activation="relu"
    ),
```

```
    # First pooling
    layers.MaxPooling2D(
        pool_size=(2,2)
    ),
```

```
    # Second convolution
    layers.Conv2D(
        64,
        kernel_size=(3,3),
        activation="relu"
    ),
```

```
    # Second pooling
    layers.MaxPooling2D(
```

```
pool_size=(2,2)
),

# Convert feature maps into vector
layers.Flatten(),

# Reduce overfitting
layers.Dropout(
    0.5
),

# Classification output
layers.Dense(
    num_classes,
    activation="softmax"
)
]
)
```

```
# -----
# Explain Architecture
# -----
```

```
divider()
print("""
        CNN PROCESSING PIPELINE
```

```

    MNIST IMAGE INPUT
    28 x 28 x 1
```



```

    Conv2D
    32 Filters
    Learns edges & shapes
```



```

    MaxPooling
```

```

    Reduces image size
    Keeps important features
```



```

    Conv2D
    64 Filters
    Learns complex patterns
```



```

    MaxPooling
    Compresses features
```



```

    Flatten
    Converts features into
    classification vector
```



```

    Dropout 50%
    Prevents memorization
```



```

    Softmax Output
    10 Digit Probabilities
```

```

    """)
```

```
# -----
# Model Summary
# -----
```

```
divider()
print("MODEL BLUEPRINT GENERATED")
```

```
print("Displaying CNN component inventory...\n")
```

```
model.summary()
```

```
divider()
```

```
print("""
CNN ARCHITECTURE COMPLETE
```

```
The network can now:
```

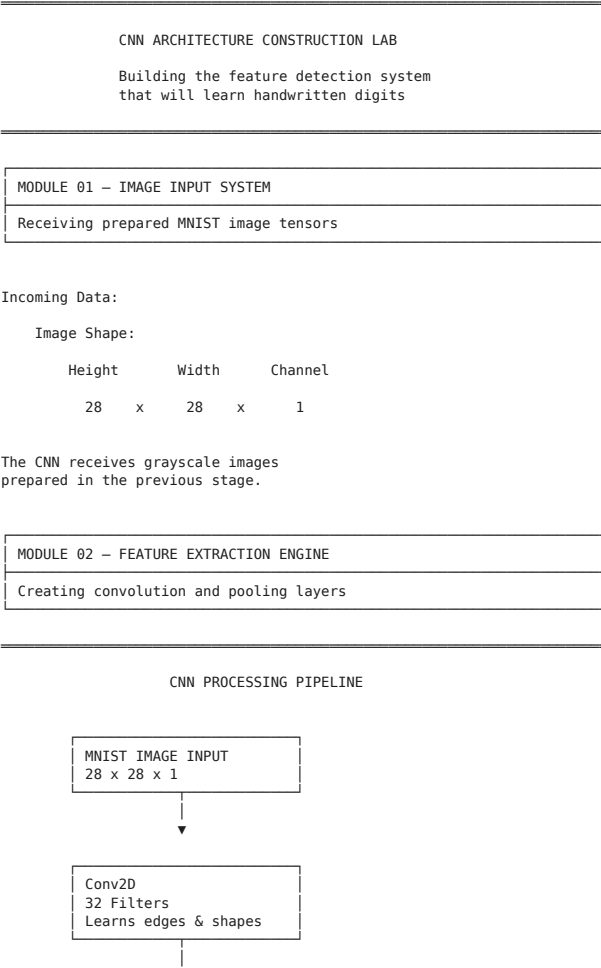
```

    Receive images
    ↓
    Extract visual features
    ↓
    Reduce unnecessary information
    ↓
    Classify digits 0-9
```

```

Next Stage:
Training the CNN and adjusting its learned weights.
""")
```

```
divider()
```

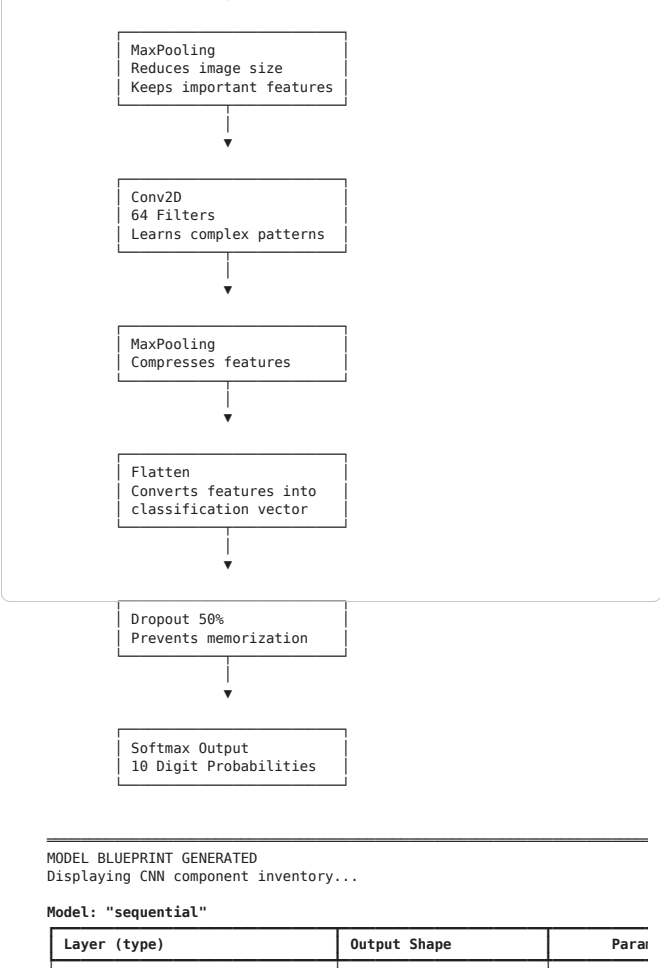


CNN PROCESSING PIPELINE

MNIST IMAGE INPUT
28 x 28 x 1

↓

Conv2D
32 Filters
Learns edges & shapes



conv2d (Conv2D)

(None, 26, 26, 32)

:

max_pooling2d (MaxPooling2D)

(None, 13, 13, 32)

:

keras.Input(shape=(28, 28, 1))

: This is the input layer of our model. It specifies the shape of the input images, which are 28x28 pixels with 1 channel (grayscale).

conv2d_1 (Conv2D)

(None, 11, 11, 64)

:

max_pooling2d_1 (MaxPooling2D)

(None, 5, 5, 64)

:

layers.Conv2D(32, kernel_size=(3, 3), activation="relu")

: This is a 2D Convolutional layer. It takes 28x28x1 input and produces 26x26x32 output. The kernel_size=(3, 3) defines the size of the convolutional filter used to detect specific features in the input image.

dense_1 (Dense)

(None, 1600)

:

dense_2 (Dense)

(None, 10)

:

Total kernel_size=(3, 3) defines the size of the convolutional filter used to detect specific features in the input image.

Trainable params: 34,826 (136.04 KB)

Non-trainable params: 0 (0.00 B)

activation="relu"

: ReLU (Rectified Linear Unit) is the activation function. It introduces non-linearity into the model, allowing it to learn complex patterns.

CNN ARCHITECTURE COMPLETE

The network can now:

layers.MaxPooling2D(pool_size=(2, 2))

: This is a Max Pooling layer. It takes 28x28x1 input and produces 13x13x1 output. The pool_size=(2, 2) reduces the spatial dimensions of the feature maps by taking the maximum value within each 2x2 window. This helps to reduce the number of parameters, control overfitting, and make the model more robust to small shifts and distortions in the input.

layers.Flatten()

: This layer flattens the 2D feature maps from the convolutional and pooling layers into a 1D vector. This is necessary to connect the convolutional part of the network to the fully connected (Dense) layers.

layers.Dropout(0.5)

: This is a Dropout layer. It randomly sets 50% of the input units to 0 at each update. This is a regularization technique that helps to prevent overfitting.

layers.Dense(num_classes, activation="softmax")

: This is the output layer. It takes a 1D vector of size 1600 and produces a vector of size 10. The num_classes is set to 10 because we have 10 classes (digits 0-9). The activation="softmax" ensures that the output values are probabilities, and they sum up to 1 across all classes. The output will be a vector of 10 probabilities, where each probability represents the model's confidence that the input image belongs to that specific digit class.

Before we can train our model, we need to compile it. Compilation involves choosing an optimizer, a loss function, and metrics to evaluate the model's performance.

- Instructions:
- Examine the code cell below.** You need to fill in the blanks for the `loss` and `optimizer` parameters in `model.compile()`.
 - Choose an appropriate loss function and optimizer** for this multi-class classification problem.
 - In the Markdown cell after the code, explain your choices.** Why are these choices suitable for this task?

```
# Cell 4: Model Compilation
# CNN Learning Configuration Center

def divider():
    print("=" * 76)

def module(title, description):
    print(f"""
{title:<74} |
{description:<74} |
""")

divider()

print("""
CNN LEARNING CONFIGURATION CENTER

Preparing the model's training instructions

""")

divider()

# -----
# Learning Strategy Explanation
# -----

module(
    "SYSTEM 01 – ERROR MEASUREMENT",
    "Selecting the method used to calculate prediction mistakes"
```

```
)

print("""
LOSS FUNCTION SELECTED:

    categorical_crossentropy

Purpose:

The CNN predicts probabilities for 10 possible digits:

Example:

Digit Prediction:

0 → 0.01
1 → 0.02
2 → 0.03
3 → 0.90 <--- Highest confidence
4 → 0.01
...

Categorical cross-entropy compares:

    Model Prediction

        VS

    Correct Answer

The goal:

Reduce the difference between
predicted probabilities and the true label.
""")

# -----
# Optimizer Explanation
# -----

module(
    "SYSTEM 02 – WEIGHT ADJUSTMENT ENGINE",
    "Choosing how the CNN improves after mistakes"
)
```

```
Correct Predictions
-----
Total Predictions

Example:

9500 correct predictions
out of 10000 images

Accuracy = 95%

This gives a simple measurement
of classification performance.
""")

# -----
# Compile Model
# -----

divider()

print("""
    ACTIVATING TRAINING SETTINGS...

Loss Function:
    categorical_crossentropy

Optimizer:
    Adam

Metric:
    Accuracy

""")

model.compile(
    loss="categorical_crossentropy",
    optimizer="adam",
    metrics=["accuracy"]
)

divider()
```

```
print("""
OPTIMIZER SELECTED:

    Adam

Purpose:

Adam updates the CNN's internal weights
after each training step.

Learning cycle:

    Image
    |
    ↓
    Prediction
    |
    ↓
    Calculate Error
    |
    ↓
    Adam Adjusts Weights
    |
    ↓
    Improved Prediction

Adam is commonly used because it combines
adaptive learning rates with efficient updates.
""")

# -----
# Metric Explanation
# -----

module(
    "SYSTEM 03 – PERFORMANCE MONITOR",
    "Tracking how often the CNN predicts correctly"
)

print("""
METRIC SELECTED:

    accuracy

Accuracy measures:
```

```
print("""
    MODEL CONFIGURATION COMPLETE

The CNN now knows:

✓ How to measure mistakes
✓ How to correct itself
✓ How success will be evaluated

Next Stage:

Training begins as the CNN learns
from thousands of handwritten examples.
""")

divider()
```

Metric:
Accuracy

MODEL CONFIGURATION COMPLETE

The CNN now knows:

- ✓ How to measure mistakes
- ✓ How to correct itself
- ✓ How success will be evaluated

Next Stage:

Training begins as the CNN learns from thousands of handwritten examples.

Explanation of Choices (To be filled by students in the reflection section):

- **Loss Function:** You need to choose a loss function that is appropriate for multi-class classification. Think about what kind of error we are trying to minimize when classifying digits into 10 categories.
- **Optimizer:** You need to choose an optimizer that will efficiently update the model's weights to minimize the loss function. Consider common optimizers used in deep learning.
- **Metrics:** We are using "accuracy" as a metric to evaluate the model's performance. Accuracy is a common metric for classification tasks, representing the percentage of correctly classified images.

Section 5: Model Training - Fitting the Model to the Data

Now it's time to train our CNN model using the training data. Training involves feeding the training data to the model and adjusting its weights to minimize the loss function.

Instructions:

1. **Examine the code cell below.** You need to fill in the blanks for `batch_size` and `epochs` in `model.fit()`.
2. **Choose appropriate values for `batch_size` and `epochs`.**

3. **Run the code cell to start training.** Observe the training progress, especially the loss and accuracy on both the training and validation sets.
4. **Experiment!** Change the `batch_size` and `epochs` and see how it affects the training process and the final performance.

```
# Cell 5: Model Training
# CNN Training Simulation Center

import time

def divider():
    print("=" * 76)

def module(title, description):
    print(f"""
{title:<74} |
{description:<74} |
""")

divider()

print("""
                CNN TRAINING SIMULATION CENTER

                The neural network begins learning from examples

""")

divider()

# -----
# Training Explanation
# -----

module(
    "TRAINING SYSTEM - LEARNING LOOP",
    "Feeding examples through the CNN and improving predictions"
)

print("""
```

TRAINING PROCESS:

```
Handwritten Image
|
↓
CNN Feature Extraction
|
↓
Digit Prediction
|
↓
Compare With Correct Answer
|
↓
Calculate Error
|
↓
Update Network Weights
|
↓
Repeat Until Improved
```

The CNN does not memorize images.
It learns patterns that help classify new handwritten digits.

```
"""
# -----
# Batch Size Explanation
# -----

module(
    "PARAMETER 01 - BATCH SIZE",
    "Controlling how many images are processed at one time"
)

batch_size = 128

print(f"""
CURRENT SETTING:
```

Batch Size = {batch_size}

Meaning:

The CNN processes {batch_size} images before updating its weights.

Training Example:

60,000 images

60,000 ÷ 128

≈ 469 learning updates per epoch

Larger batches:

- + Faster processing
- More memory usage

Smaller batches:

- + More frequent updates
- More training noise

```
""")

# -----
# Epoch Explanation
# -----

module(
    "PARAMETER 02 - EPOCHS",
    "Determining how many times the CNN reviews the dataset"
)

epochs = 15

print(f"""
CURRENT SETTING:
```

Epochs = {epochs}

One epoch means:

The CNN sees the entire training dataset once.

Current mission:

The CNN will study the handwritten digits
{epochs} complete times.
"""

Validation Explanation

```
module(
    "PARAMETER 03 – VALIDATION CHECKPOINT",
    "Testing learning progress during training"
)
```

```
print("""
Validation Split:

    10% of training images are separated
    and used as a progress checkpoint.
```

Purpose:

Training Data:

Used for learning

Validation Data:

Used for checking if learning
generalizes to unseen examples

This helps identify:

- ✓ Improving performance
- △ Possible overfitting

```
"""
```

Start Training

- Convolution filters
- Feature detectors
- Classification weights

Next Stage:

Evaluate performance on unseen test images.
"""

divider()

```
divider()

print("""
    TRAINING SEQUENCE STARTING
```

Batch Size:
128 images

Epoch Count:
15 training cycles

Validation:
10% checkpoint enabled

The CNN is now beginning to learn...
"""

divider()

```
history = model.fit(
    x_train,
    y_train,
    batch_size=batch_size,
    epochs=epochs,
    validation_split=0.1
)
```

Training Complete

```
divider()

print("""
    TRAINING COMPLETE
```

The CNN has completed its learning cycles.

The model has now adjusted:

The CNN has completed its learning cycles.

The model has now adjusted:

- Convolution filters
- Feature detectors
- Classification weights

Next Stage:

Evaluate performance on unseen test images.

Explanation of Training Parameters:

- **batch_size**: This determines the number of training samples processed in each mini-batch during training. A larger batch size can speed up training but might require more memory. A smaller batch size can lead to more noisy updates but might generalize better.
- **epochs**: One epoch represents one complete pass through the entire training dataset. More epochs can potentially lead to better training but also increase the risk of overfitting, where the model learns the training data too well and performs poorly on unseen data.
- **validation_split=0.1**: This reserves 10% of the training data as a validation set. During training, the model's performance is evaluated on this validation set after each epoch. This helps to monitor for overfitting and tune hyperparameters.

Section 6: Model Evaluation - Assessing Performance on Test Data

After training, we need to evaluate our model's performance on the test dataset. This gives us an estimate of how well the model generalizes to unseen data.

Instructions:

1. **Run the code cell below.**
2. **Observe the output.** It will print the test loss and test accuracy.
3. **Think about the results.** Is the test accuracy satisfactory? How does it compare to the training and validation accuracy you observed during training?

Cell 6: Model Evaluation
CNN Evaluation Laboratory

```
import numpy as np

def divider():
    print("=" * 76)

def module(title, description):
    print(f"""
{title:<74} |
{description:<74} |
""")

divider()

print("""
        CNN EVALUATION LABORATORY

        Testing the model on unseen handwritten digits

""")

divider()

# -----
# Evaluation Explanation
# -----

module(
    "EVALUATION SYSTEM – GENERALIZATION TEST",
    "Measuring how well the CNN performs on new examples"
)

print("""
The CNN has completed training.

Training Data:

        Used to adjust model weights

Test Data:
""")
```

```
        against correct answers...

""")

divider()

loss, accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

# -----
# Results Display
# -----

divider()

print("""
        FINAL PERFORMANCE REPORT

""")

print(f"""
{
    Evaluation Metric
    Test Loss:      {loss:.4f}
    Test Accuracy: {accuracy:.4f}
}
""")

print(f"""
Accuracy Percentage:

        {accuracy * 100:.2f}%

Interpretation:

The CNN correctly classified approximately
{accuracy * 100:.2f}% of unseen handwritten
digits from the MNIST test dataset.
""")
```

```
Never shown during training

Purpose:

Determine whether the CNN learned
general patterns instead of memorizing
the training images.
""")

# -----
# Test Dataset Information
# -----

module(
    "TEST DATABASE – MNIST EXAMINATION SET",
    "Providing unseen handwritten examples"
)

print(f"""
Test Images:

        {x_test.shape[0]:,} handwritten digits

Image Format:

        Height: {x_test.shape[1]} pixels
        Width : {x_test.shape[2]} pixels
        Channel: {x_test.shape[3]}

The CNN must classify each image
without additional training.
""")

# -----
# Run Evaluation
# -----

divider()

print("""
        RUNNING CNN PERFORMANCE CHECK

        Comparing predictions
""")
```

```
# -----
# Final Mission Report
# -----

divider()

print("""
        CNN PROJECT COMPLETE

Pipeline Summary:

📦 Dataset Collection
    ↓
📁 Data Preparation
    ↓
🏗️ CNN Architecture
    ↓
⚙️ Learning Configuration
    ↓
🧠 Model Training
    ↓
📊 Performance Evaluation

The CNN has successfully transformed
pixel information into digit predictions.
""")

divider()
```

Accuracy Percentage:

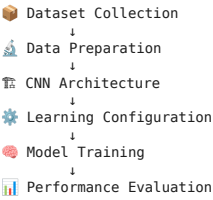
99.07%

Interpretation:

The CNN correctly classified approximately 99.07% of unseen handwritten digits from the MNIST test dataset.

CNN PROJECT COMPLETE

Pipeline Summary:



The CNN has successfully transformed pixel information into digit predictions.

CNN Reflection Data Collector - Architecture